

Bombino Express — Final Phase Product Brief

For: Aditya

From: Arbaaz

Purpose: Build a project plan to deliver this in **3 weeks**.

Reference build: The approved demo

(<https://demo-bombino-express.vercel.app/>) is the visual + UX source of truth for every screen and flow described here.

1. What this is

The final phase turns Bombino Express from a **single-customer booking app** into a **full self-service courier platform** with three connected experiences in one product:

1. **Customer** — signs up themselves, books door-to-door pickups, pays, and tracks.
2. **Pickup Agent** — a field-staff mobile experience for collecting parcels.
3. **Operations / Admin** — an internal console that runs every order through its full lifecycle until the shipment goes live.

A working demo of all of this has already been built and approved by the client. **Your job is to make it real.**

2. The single most important thing to understand

The demo is a stage set. It looks complete, but there is nothing behind it.

The approved demo is **frontend-only**. Every API call is faked, there is no real backend, no real ITD integration, no real authentication, no database persistence, no payment gateway. It exists to show the client the experience.

This phase is about building everything behind those screens for real:

The demo fakes...	The real build needs...
-------------------	-------------------------

Hardcoded mock API responses	Real backend endpoints + database persistence
A role toggle in the menu	Real role-based access control (RBAC) with enforced permissions
OTP that accepts any code	A real SMS/OTP provider
Account creation that just sets local state	Real account creation, incl. the ITD <code>add_customer</code> API for corporates
Payments that always "succeed"	A real payment gateway (Razorpay)
An AWB that's generated locally	A real ITD docket + AWB via the <code>create_docket</code> API
Status changes held in browser memory	Real, persisted order state that survives refresh and is shared across users

Reuse the demo's UI and flows. Rebuild everything underneath as production code on the real app codebase.

3. Core architectural constraint (read before planning anything)

ITD dockets cannot be amended after creation. Once a docket + AWB is created in ITD, it is frozen — the weight, charges, and everything else are locked.

This forces the **deferred-docket model**, which is the backbone of the entire operations flow:

- **Booking does NOT create an ITD docket.** It creates a **Bombino order** with an internal Order ID.
- The order lives inside the **operations console** through its entire pre-transit life (pickup → weighing → payment settlement).
- The **real ITD docket + AWB is created only once, at the very end**, after the parcel is physically weighed and payment is settled, using the final confirmed values.
- Only at that point does the customer get a real AWB and live tracking.

The operations console is therefore the system of record for every order before it becomes a shipment. This is not optional — it's the only way the model works given ITD's constraint.

4. Account Creation

Single "Create account" entry → a chooser screen → **Personal** or **Company**. The two paths diverge from there.

4.1 Personal accounts

- **Mobile number + OTP verification** (real OTP provider).
- Collect: full name, email, phone, address.
- **Aadhaar: collect at signup — number + optional document upload — but format-validate only.** No live/paid Aadhaar verification API in this phase. (Store it; ITD's own KYC flag governs whether it's needed at booking.)
- Personal customers **all ride a single shared retail ITD account** (credentials provided by Bombino), but are **individuated on our side** via our own customer records. Each personal user is a distinct Bombino account even though ITD sees one shared account underneath.

4.2 Company accounts

- **Mobile number + OTP verification** (real OTP provider).
- Collect company details + **GST number (format-validate only — no live GST registry API in this phase).**
- On completion, **create a real ITD customer at signup** via the ITD `add_customer` API (see §4.3).

4.3 ITD `add_customer` API (corporate accounts)

Endpoint: `POST`

`https://admin.bombinoexp.com/api/customer_api/add_customer`

Content-Type: `application/x-www-form-urlencoded`, Bearer auth.

Fields:

Field	Source	Notes
<code>name</code>	Company name	
<code>customer_country</code>	e.g. <code>IN</code>	

location_code	⚠ Pending ITD	Confirm with Anas — same as hub or separate?
customer_category	Company	Fixed for corporate
email	Signup	
contact_no	Signup	
contact_no2	Optional	
address, pincode, city, state	Signup	
contact_person	Signup	
hub_id	⚠ Pending ITD/Bombino	How to assign — dropdown, pincode-derived, or default? Which hubs to expose?
company_id	⚠ Pending ITD	Likely 2 for Bombino — confirming with Anas
gst_number	Signup	
customer_setting	JSON, e.g. {"show_kyc_after_login":1, "is_fsc_apply":1}	ITD carries its own KYC flag here

⚠ **Three fields are pending ITD confirmation (hub_id, company_id, location_code).** These are **config values, not architecture**. Build the flow to accept them as configurable inputs; the exact values drop in when Anas confirms. **Do not let these block the build** — treat them as constants/config you can set later.



Hub reference list (24 hubs, ID at end): MUMBAI-1, HYDERABAD-2, DELHI-3, CHANDIGARH-4, AHMEDABAD-5, BANGALORE-6, PUNE-7, FORT OFFICE-8, JAIPUR-9, CHENNAI-10, KOLKATA-11, LOWER PAREL-12, LONDON-13, NEW YORK-14, UK BRANCH-15, MAIL BOX MART-16, MANHATTAN-17, OZONE PARK-18, BOMBINO DUBAI-19, DUBAI BRANCH-20, AUSTRALIA-21, SURAT-23, SINGAPORE-24, KENYA-25.

4.4 Post-signup

- After signup, the user lands logged in and proceeds into the booking flow.
- **KYC is captured once at onboarding and not asked again during booking** (both personal and corporate book without a repeated KYC step).

5. Customer Journey (booking)

Reuse the existing polished booking flow. Add:

- **Pickup vs. drop-off toggle** at booking. If pickup: **date + time-slot selection** (four 3-hour windows, 9 AM–9 PM). Maps to ITD's `pickup_request` param (1 = pickup, 2 = drop-off).
- **Payment options at booking** (see §7).
- Booking ends on an **Order ID** success state (NOT an AWB — the AWB comes later from ops).
- The booked order carries everything ops needs downstream: consignee, origin/pickup address, real amount, pickup choice + slot, payment method, booked weight.

6. Operations Console (the core of this phase)

The internal surface where the team runs every order. Reuse the demo's console design.

6.1 The order board

- Orders grouped by **lifecycle phase** into **4–5 columns** (e.g. Pickup → At Branch → Settle → Dispatch → Done). Kanban on desktop; a clean grouped list on mobile.
- Legacy/other orders bucketed separately and collapsed by default.

6.2 The full order lifecycle

Each order advances through named actions (each is a real, persisted state change):

Pickup path: Pickup requested → Agent accepts → Out for pickup → Picked up → Received at hub
Drop-off path: Awaiting drop-off → Received at hub
Both, at hub onward: Weigh → (Collect payment if due) → Settle → Ready for docket → **Generate docket & AWB**

6.3 Weigh + settle (the critical reconciliation beat)

- At the hub, staff enter the **actual weighed weight**.
- System compares against the **booked (estimated) weight** and **recalculates the final amount**.
- Show the delta clearly: **collect more / refund due / no change**.
- The order cannot proceed to docket until weight is captured and payment is satisfied.

6.4 Docket & AWB generation (the finale)

- Only after weight + payment are settled, call the **ITD create_docket API** with the **final** values to generate the real docket + AWB.
- The Order ID now maps to a real AWB; the customer's order becomes trackable.
- ⚠️ **Note:** `create_docket` had an open issue around the freight amount / `api_service_code` / weight unit (kg vs lb). Confirm the current working payload with Anas before wiring this — it must produce valid AWBs reliably.

7. Payment Model (Bombino-side only — ITD does NOT handle payment)

Four methods, selected at booking:

Method	When collected	By whom
Pay now (advance)	At booking	Payment gateway (Razorpay)
Pay at pickup	At the doorstep	Pickup agent marks collected
Pay at drop-off	At the hub	Admin marks collected
Pay at delivery (COD)	At destination — recipient pays	Not collected by us — flag only, for the accounts team

Key rules:

- Advance uses a **real payment gateway** (Razorpay or as decided). All others are collected physically and **marked** in the system.
- Because the amount is an estimate until weighing, **advance payments still reconcile at pickup/hub** — collect-more or refund is handled at the settle step.
- **COD must not block the docket** — it's a flag; the order settles and dockets freely, just marked COD for accounts.
- Payment is **entirely Bombino-side** — no payment fields go to ITD.

8. Roles & Access (real RBAC — not a demo toggle)

Three roles, with **enforced, backend-checked permissions**:

1. **Customer** — customer app only. Cannot see or reach any ops/agent screens or APIs.
2. **Pickup Agent** — field mobile view: available pickups (self-accept), my pickups, and field actions (accept → out for pickup → collect payment if pay-at-pickup → picked up → received at hub).
3. **Operations / Admin** — the full console: hub actions from weigh → settle → docket. (A **Super Admin** tier that manages internal users/roles is desirable — see §11.)

Requirements:

- Roles stored on the user record; permissions enforced **on the backend**, not just hidden in the UI.
- Admin APIs reject non-admin users (403). Customers hitting an admin route/API are blocked.
- Pickup agents **self-accept** jobs (no admin-assignment bottleneck).
- Clean handoff: once an order is "received at hub," it leaves the agent's queue and admin takes over.

9. Live status sync to the customer

Every ops/agent action updates the customer-facing status in real time:

Internal action	Customer sees
Agent accepts	Pickup confirmed
Out for pickup	Agent on the way

Picked up	Parcel picked up
Received at hub	Arrived at Bombino hub
Weigh / settle / ready	(no customer change — internal)
Docket + AWB generated	In transit (+ becomes trackable)

Changes must reflect **across all views** (customer, agent, admin) — this is real shared state, not per-browser memory. Persisted so a refresh shows current status.

10. Out of scope for this phase (do NOT build)

These were discussed but are **explicitly excluded** from the 3-week scope. Do not build them:

- **Live map / real-time GPS pickup tracking** (Swiggy-style). Status-update notifications only.
- **Voice AI agent.**
- **Live/paid Aadhaar KYC verification API.**
- **Live/paid GST registry verification API.**
- **Master tracking / US-airport segregation.**

If any of these come up, flag to Arbaaz — they are future paid phases, not this one.

11. Open items / pending confirmations

- **⚠ ITD fields (**hub_id**, **company_id**, **location_code**)** — pending Anas. Build as configurable; don't block on them.
- **Hub assignment logic** — how a new customer's hub is chosen (dropdown / pincode-derived / default) and which hubs to expose — pending Bombino + ITD.
- **create_docket payload** — confirm the working freight / **api_service_code** / weight-unit setup with Anas before wiring the finale.
- **Payment gateway** — confirm Razorpay (or alternative) and credentials.
- **OTP provider** — confirm which SMS/OTP service.
- **Super Admin / internal-user management** — confirm whether in-scope for this phase or handled manually for now.

12. How to approach the plan

- **The demo is your reference for every screen and flow** — match its UX; build the real backend behind it.
- **Suggested sequencing** (yours to refine): (1) real auth + RBAC foundation, (2) account creation incl. ITD `add_customer`, (3) booking with pickup + payment, (4) the ops lifecycle engine + persistence, (5) pickup agent flow, (6) `create_docket` finale + live tracking, (7) payment gateway, (8) status sync, (9) end-to-end testing.
- **Pending ITD fields do not block you** — they're config.
- Target: **working, tested, end-to-end in 3 weeks.**

Draft your project plan around this and share it back with a proposed week-by-week breakdown and any blockers you foresee.